

Project 1 Writeup

Method

I used a local descent method with backtracking line search for all three simple problems. At each iteration, the optimizer computes the gradient, forms the normalized descent direction

$$d = -\frac{\nabla f(x)}{\|\nabla f(x)\|},$$

Then, the method uses backtracking line search to choose the step size if:

$$f(x^{(k+1)}) \leq f(x^{(k)}) + \beta \alpha \nabla_{d^{(k)}} f(x^{(k)})$$

with initial params $\alpha = 1.0$, $\beta = 1e - 4$ and a constant factor $p = 0.5$ to decrease α until sufficient decrease is satisfied.

The update then is

$$x_{k+1} = x_k + \alpha_k d_k.$$

The implementation checks the remaining budget before taking a new step.

Local Test Results

Current implementation beat random search at the following rates:

- **simple1**: 71.8%
- **simple2**: 96.6.0%
- **simple3**: 99.0%

This method is simple and reliable on the provided test functions. It performs best on Himmelblau's and Powell's functions.

The performance on **simple1** could be due to Rosenbrock's function being more difficult for normalized gradient descent because the optimum lies inside a curved valley. The method tends to zig-zag across that valley, and with only 20 evaluations available, there is limited budget to correct the direction and take many productive steps.

Rosenbrock Path

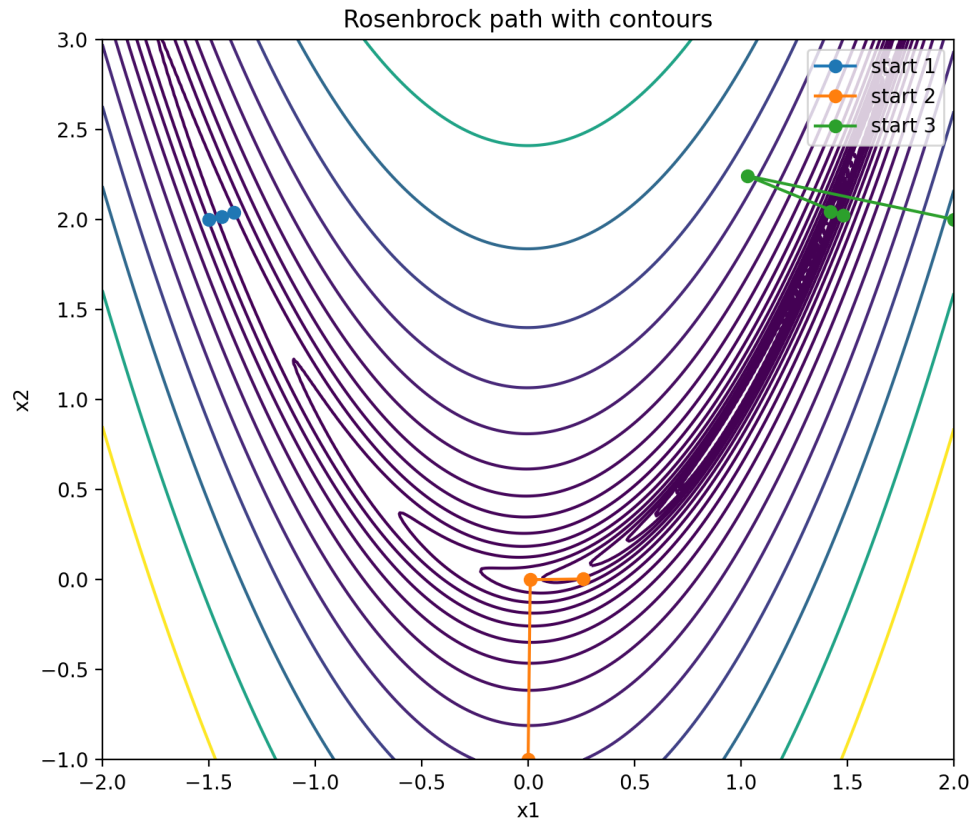
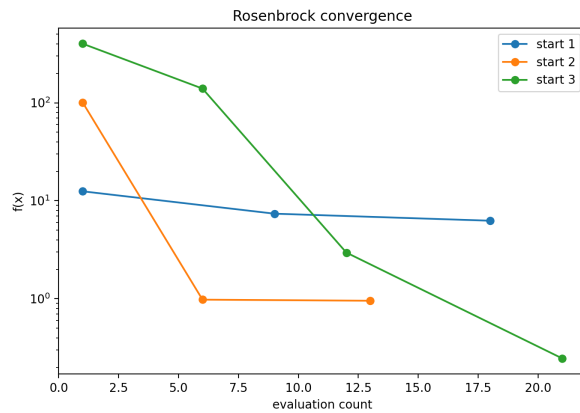
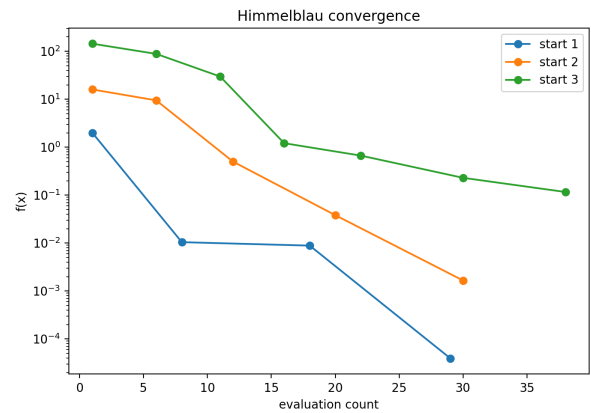


Figure 1: Optimization paths on Rosenbrock's function from three initial points, overlaid on objective contours.

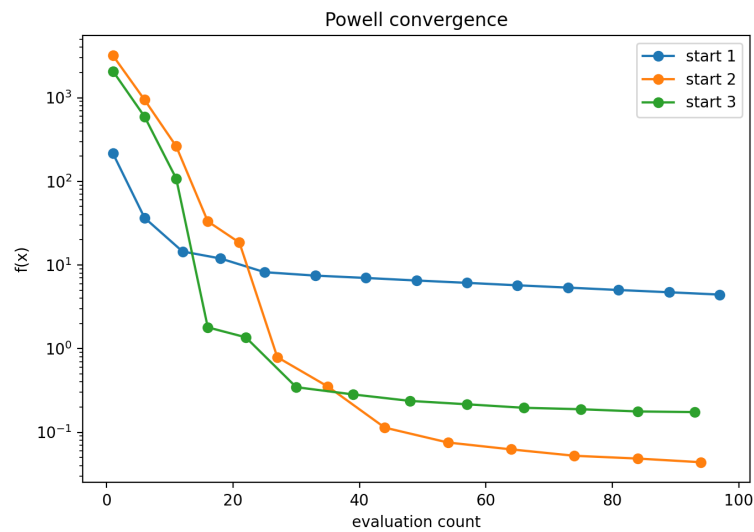
Convergence Plots



(a) Rosenbrock



(b) Himmelblau



(c) Powell

Figure 2: Convergence behavior for the three simple optimization problems.

Relevant Code

```

1 def descent_step(f, g, x, n, count):
2     if count() + 3 > n:
3         return x, False
4
5     g_x = g(x)
6     f_x = f(x)
7     g_norm = np.linalg.norm(g_x)
8
9     if (not np.isfinite(g_norm)) or g_norm == 0.0:
10         return x, False
11

```

```
12     d = -g_x / g_norm
13
14     alpha = 1.0
15     p = 0.5
16     beta = 1e-4
17
18     while count() < n:
19         x_trial = x + alpha * d
20         f_trial = f(x_trial)
21         if f_trial <= f_x + beta * alpha * (g_x.T @ d):
22             return x_trial, True
23         alpha *= p
24
25     return x, False
26
27
28 def optimize(f, g, x0, n, count, prob):
29     x_best = x0
30
31     while count() < n:
32         x_new, moved = descent_step(f, g, x_best, n, count)
33         if not moved:
34             break
35         x_best = x_new
36
37     return x_best
```